Assignment: LLM Inference on Mobile WebGPU

Profiling and Custom GEMM Kernels for Gemma E2B on Adreno

Target model	Gemma E2B (Gemma 3n)
Target hardware	Snapdragon device with Adreno GPU
Runtime	Chrome for Android, WebGPU
Deliverables	Part 1: profiling report. Part 2: custom GEMM kernel, comparison table, benchmark harness

Part 1: Baseline and Profiling (no optimization)

Task. Run Gemma E2B on an existing WebGPU stack, such as WebLLM, MediaPipe LLM Inference, or Transformers.js. If E2B is not supported out of the box, use the closest Gemma variant and document why. Do not optimize anything in this part.

Metrics to record

Run each metric at least 5 times and report the median and p95.

#	Metric	Notes
1	Tokens/sec	Decode throughput
2	Prefill latency	Fixed prompts of 128, 512, and 1024 tokens
3	Decode latency per token	Report distribution, not just mean
4	Memory usage	GPU buffers, JS heap, device RSS
5	Kernel (dispatch) count per token	Count from the command stream
6	Individual kernel execution time	Per-kernel table, grouped by op type

Profiling methods

- WebGPU `timestamp-query` for per-pass GPU timing.
- Chrome tracing and `chrome://gpu` for the pipeline view.
- Snapdragon Profiler for hardware counters, if available.
- `adb shell dumpsys meminfo` for memory.

Analysis

Identify the **top 2-3 bottleneck kernels**. Then classify the dominant bottleneck as one or more of: memory bandwidth, compute, synchronization, kernel launch overhead, dequantization, poor occupancy. Justify each classification with evidence.

Classification	Example evidence
Memory bandwidth	Achieved GB/s versus device peak; arithmetic intensity on a roofline
Compute	Achieved GFLOPS versus peak FP16/FP32 throughput
Synchronization	Barrier stalls; gaps between passes
Launch overhead	Wall time per token versus the sum of kernel times; many tiny dispatches

Classification	Example evidence
Dequantization	Time share of unpack/scale ops; compare against an FP16-weight run
Poor occupancy	Low wave counts; register or shared-memory pressure in counters

Deliverable

- A profiling report with a per-kernel time table.
- A stacked time-breakdown chart.
- The bottleneck classification with supporting evidence.
- A description of your methodology, including device, driver, Chrome version, and thermal state.

Part 2: Custom GEMM Kernel

Pipeline. Q4/Q8 weights, then dequantization, then matrix multiplication, then FP16 accumulation.

Kernel	Description
A	Baseline naive WGSL
B	Tiled GEMM using workgroup shared memory
C	Tiled with vectorized loads (`vec4`, packed `u32`)
D	Fused dequant + GEMM

Requirements

- Benchmark each kernel in isolation on Gemma's real layer shapes: the decode GEMV (M=1) and prefill (M>=128).
- Sweep tile sizes and workgroup sizes.
- Report GFLOPS, effective GB/s, and correctness (max absolute error against an FP32 reference).
- Check support for the `shader-f16` feature and provide a fallback if it is absent.
- Use proper timing: warm-up runs, many iterations, timestamp-query rather than CPU timers.

Deliverable

- A custom GEMM kernel that demonstrably beats the baseline GEMM on your target Adreno.
- A comparison table for kernels A-D.
- An explanation of why each variant is faster or slower.
- A reproducible benchmark harness (code, instructions, and raw results).

Suggested results table

Kernel	Shape (M,N,K)	Tile / WG	Time (ms)	GFLOPS	GB/s	Max abs err
A						
B						
C						
D						

Grading Rubric (suggested)

Item	Weight
Profiling rigor and methodology	30%
Bottleneck analysis quality	20%
Kernel correctness	15%
Measured speedup	25%
Report clarity and reproducibility	10%

References

[1] W3C, *WebGPU Shading Language (WGSL)*, CRD 2026-07-03: https://www.w3.org/TR/2026/CRD-WGSL-20260703/

[2] W3C, *WebGPU* specification (includes timestamp-query, shader-f16): https://www.w3.org/TR/webgpu/

[3] MLC AI, *WebLLM*: https://github.com/mlc-ai/web-llm

[4] MLC AI, *MLC-LLM*: https://github.com/mlc-ai/mlc-llm

[5] Google, *Gemma 3n documentation*: https://ai.google.dev/gemma/docs/gemma-3n

[6] Google AI Edge, *LLM Inference API (MediaPipe)*: https://ai.google.dev/edge/mediapipe/solutions/genai/llm_inference

[7] Hugging Face, *Transformers.js*: https://github.com/huggingface/transformers.js

[8] ggml-org, *llama.cpp* (Q4_0/Q8_0 block formats, Adreno OpenCL backend): https://github.com/ggml-org/llama.cpp

[9] Qualcomm, *Snapdragon Profiler* and Adreno developer resources: https://www.qualcomm.com/developer

[10] Chrome for Developers, *WebGPU*: https://developer.chrome.com/docs/web-platform/webgpu

Note: verify all links before distribution.